

Vasic-Digital-Reusable-Module-Suite

Construye una vez, reutiliza en todas partes — una flota de módulos Go y KMP pequeños, desacoplados y probados de forma independiente.

Resumen

Una amplia familia de módulos genéricos y reutilizables publicados bajo los espacios de nombres `digital.vasic.*` (Go) y Kotlin Multiplatform. Cada módulo es independiente, probado y versionado por separado, e integrado como submódulo de código equivalente en productos más grandes (Catalogizer, HelixAgent y el resto de la flota). Esta página consolida las numerosas utilidades pequeñas que, de presentarse por separado, generarían ruido como páginas individuales.

Descripción breve

Una suite seleccionada de módulos desacoplados `digital.vasic.*`: primitivas de infraestructura (autenticación, caché, base de datos, configuración, observabilidad), bloques de construcción para AI/agentes (RAG, VectorDB, *Embeddings*, MCP, *Agentic*, *Planning*), y barreras defensivas-LLM (*RedTeam*, *Normalize*), además de un conjunto espejo en Kotlin Multiplatform. Cada uno es genérico, probado y reutilizable.

Descripción detallada

La organización *vasic-digital* opera bajo una apuesta estructural: una filosofía de “constitución + múltiples submódulos reutilizables y desacoplados” en la que la funcionalidad genérica nunca se escribe dos veces. En lugar de monolitos, cada componente reutilizable se extrae en su propio módulo pequeño —su propio repositorio, sus propias pruebas, su propia documentación— y se mantiene estrictamente desacoplado para que los detalles de ningún consumidor se filtren. Esta página los agrupa porque,

tomados de uno en uno, cada uno tiene escala de biblioteca y resultaría ruido como página de producto individual. En conjunto, sin embargo, son el verdadero multiplicador de fuerza de la organización: un activo de ingeniería privado que convierte “construir un nuevo producto” en “ensamblar piezas probadas”, y la base concreta para afirmar que esta flota no reinventa la rueda, sino que mantiene una rueda muy buena y la despliega en todas partes.

La suite abarca tres grupos. **Primitivas de infraestructura** (Go) proporcionan la base que todo servicio necesita: auth (JWT/bcrypt), cache (Redis/TTL), database (migraciones, dual SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (centro WebSocket), eventbus, filesystem (multiprotocolo), discovery/mdns, http3, recovery, concurrency, lazy, entre otros. **Bloques de construcción para AI/agentes** (Go) ofrecen el sustrato para sistemas AI: rag, vectordb, embeddings, memory, conversation (compresión de contexto infinito, *event sourcing*), mcp (Model Context Protocol), toolschema, skillregistry, agentic (orquestración de flujos basada en grafos), planning (HiPlan/MCTS/Árbol-de-Pensamiento), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (modelado de recompensas/RLHF) y toon (*Token-Oriented Object Notation*). **Barreras defensivas-LLM** proporcionan herramientas de robustez adversarial: RedTeam (fixtures adversariales impulsados por YAML) y Normalize (canonicalización de entradas adversariales). Un conjunto paralelo en **Kotlin Multiplatform** replica módulos clave (Auth-KMP, Database-KMP, Security-KMP, Componentes-UI-KMP, etc.) para aplicaciones multiplataforma.

Por qué lo creamos

Desarrollar muchos productos (Catalogizer, HelixAgent, Herald y otros) desde cero cada vez resulta ineficiente e inconsistente. Extraer cada preocupación genérica en un módulo desacoplado y probado significa que las correcciones y mejoras se propagan a toda la flota, y cada nuevo producto se ensambla a partir de componentes ya validados.

Por qué cambia las reglas del juego

Es, en la práctica, una “biblioteca estándar” privada para construir backends centrados en AI, esa capa que la mayoría de los equipos nunca llega a desarrollar porque están demasiado ocupados resolviendo por quinta vez autenticación, almacenamiento en caché y la infraestructura básica de RAG. Aquí, los primitivos de infraestructura, los bloques de construcción de AI y los guardarraíles defensivos de LLM existen

como módulos independientes y probados, listos para integrarse. Esto permite que un equipo pequeño despliegue sistemas de nivel productivo a un ritmo que normalmente requeriría uno mucho más grande, y sin acumular la deuda de duplicación que suele surgir en el proceso.

Qué tiene de innovador

- **Disciplina de desacoplamiento a nivel de flota (CONST-051):** los submódulos se tratan como bases de código independientes, sin incluir especificidades del consumidor.
- **Capa dedicada de primitivos AI (RAG, VectorDB, Incrustaciones, MCP, ToolSchema, Agéntico, Planificación, LLMOps)** como módulos reutilizables.
- **Conjunto de guardarraíles defensivos LLM (RedTeam, Normalize)** para robustez ante ataques.
- **Conjuntos de módulos paralelos Go + Kotlin Multiplatform** que comparten las mismas convenciones.

Desafíos y soluciones

- **Evitar la degradación por acoplamiento en decenas de módulos:** resuelto con el contrato de desacoplamiento de la constitución y la inyección en tiempo de ejecución de especificidades del consumidor.
- **Mantener la coherencia y las pruebas en muchos módulos:** resuelto con una convención compartida (pruebas/docs/Desafíos por módulo) y la columna vertebral de gobernanza HelixConstitution.
- **Alcance multiplataforma:** resuelto con un espejo Kotlin Multiplatform de los módulos principales.

Tecnologías empleadas (por qué y cómo)

- **Go** — la mayoría de los módulos (digital.vasic.*).
- **Kotlin Multiplatform** — módulos espejo multiplataforma (Autenticación/Base de datos/Seguridad/Interfaz de usuario/Concurrencia/Control de tasa-KMP).
- **Redis / PostgreSQL / SQLite** — primitivos de caché, base de datos y almacenamiento.
- **Prometheus / OpenTelemetry** — módulo de observabilidad.
- **WebSocket / HTTP/3 (quic-go) / mDNS** — módulos de red.
- **Vector DB / embeddings / RAG / MCP** — módulos de primitivos AI.

- **YAML** — configuraciones y casos de prueba adversariales de RedTeam.

NO VERIFICADO / EN DESARROLLO: varios repositorios de la organización están marcados como “ESQUELETO / EN DESARROLLO” (p. ej., PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). Presentarlos como en fase inicial/esqueleto, no como productos finales.